

AIML CZG565 · ML COMPANION READER

Linear Models for Regression

From fitting a straight line to Ridge and Lasso regularisation

Module 3 · Read alongside the lecture slides · Every slide example worked in full

How to use this companion. Coloured boxes mark different kinds of help. Blue **Intuition** boxes give the plain-English idea. Amber **Everyday picture** boxes give a real-life analogy. Green **Worked example** boxes solve problems step by step with real numbers. Red **Watch out** boxes flag common mistakes. Purple **Key takeaway** boxes give the one line to remember.

1 What Is Regression?

Every day we want to predict a number. Think of a house price, the risk of heart disease, or how long a battery lasts. The input is a set of measurable features. The output is a continuous number. That task is called **regression**.

The intuition

Regression asks: “If I change the inputs, how does the output move?” It finds the rule that maps features to a number. Linear regression finds a rule that is a straight line (or a flat plane in higher dimensions).

★ Everyday picture

A property agent knows that house price rises with floor area. She draws a line through past sales data. Given a new flat’s area, she reads the price off the line. That is linear regression in everyday life.

Formally, we have n training examples. Each example has d input features, written as a vector $\mathbf{x}^{(i)} = (x_1^{(i)}, x_2^{(i)}, \dots, x_d^{(i)})$. Each example has a real-valued target $y^{(i)}$. We want a function $h(\mathbf{x})$ such that $h(\mathbf{x}^{(i)}) \approx y^{(i)}$.

✓ Key takeaway

Regression maps input features to a real number. Linear regression models that mapping as a weighted sum of the features. Next: we look at the simplest case — one feature, one line.

2 Simple Linear Regression

Simple linear regression (SLR) uses one input feature x to predict y . The model is a straight line:

$$h(x) = w_0 + w_1x.$$

Here w_0 is the **intercept** (where the line crosses the y -axis) and w_1 is the **slope** (how much y changes per unit change in x). Both are called **weights** or **parameters**. We learn them from the data.

The intuition

Picture a scatter plot of points. We want the line that sits “as close to all points as possible”. “Close” means: the vertical gap between a point and the line (the **residual**) is small.

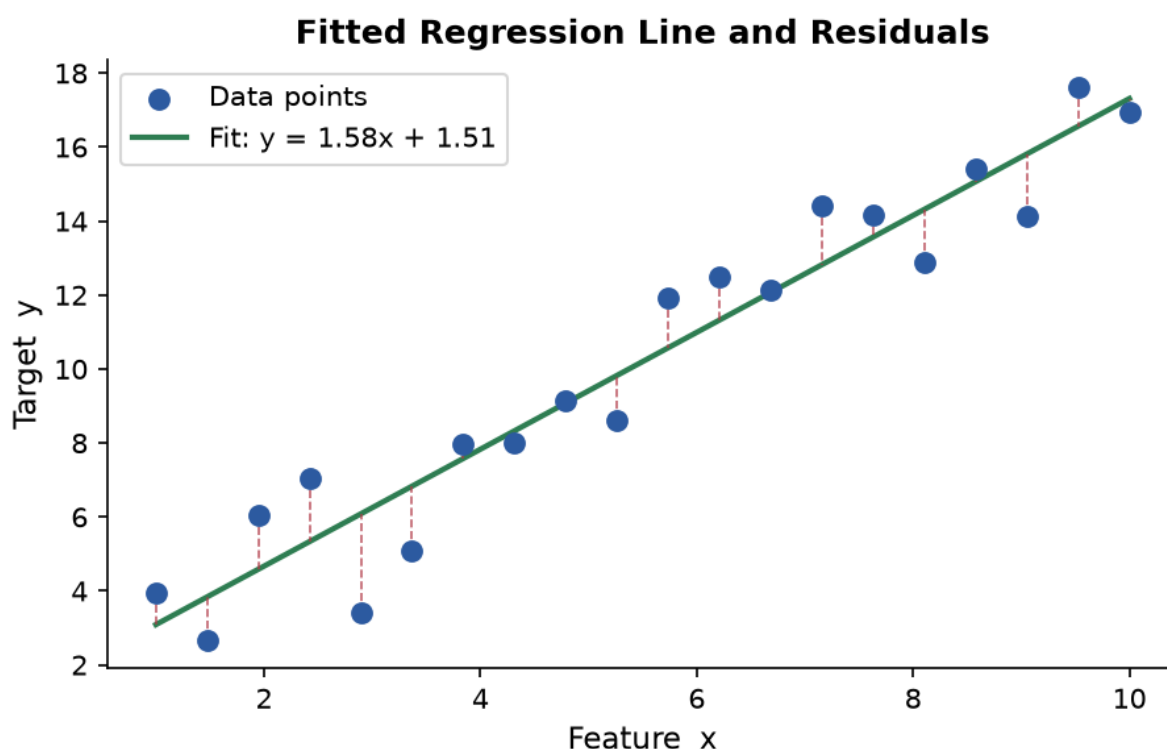


Figure 1: A scatter plot and fitted line. The red dashed verticals are the residuals — the gaps the model must minimise. The fitted line minimises the total squared residuals.

★ Everyday picture

Imagine pinning a rubber band flat across a scatter of pins on a board. The band takes the position that balances the pulls from all the pins. That is exactly what the least-squares line does.

2.1 The residual and why we square it

The **residual** for example i is the gap between prediction and truth:

$$e^{(i)} = h(x^{(i)}) - y^{(i)}.$$

A positive residual means we predicted too high; negative means too low. We want all residuals small. We cannot just add them up because positives and negatives cancel. We square each one so they are all non-negative, then add.

Worked example: Residuals for three data points

Say our line is $h(x) = 1 + 2x$. Three points: $(1, 3)$, $(2, 5)$, $(3, 6)$.

Step 1. Predict: $h(1) = 3$, $h(2) = 5$, $h(3) = 7$.

Step 2. Residuals: $e_1 = 3 - 3 = 0$, $e_2 = 5 - 5 = 0$, $e_3 = 7 - 6 = 1$.

Step 3. Squared residuals: 0, 0, 1.

Step 4. Sum of squares: $0 + 0 + 1 = 1$.

The fit is perfect for two points and off by one for the third.

✓ Key takeaway

SLR fits $h(x) = w_0 + w_1x$ to data. The residual $e^{(i)} = h(x^{(i)}) - y^{(i)}$ is the prediction error. We square residuals before summing to avoid cancellation.

3 The Loss Function: Mean Squared Error

We need a single score that says how bad our current parameters are. That score is the **loss function**. The most common choice is **mean squared error** (MSE):

$$J(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n (h(\mathbf{x}^{(i)}) - y^{(i)})^2.$$

We divide by n to get an average, so the loss does not grow just because we have more data.

The intuition

MSE is the “average squared miss distance”. A loss of zero means perfect predictions. A large loss means the predictions are far from the true values. Training finds the weights $\mathbf{w}$ that make $J(\mathbf{w})$ as small as possible.

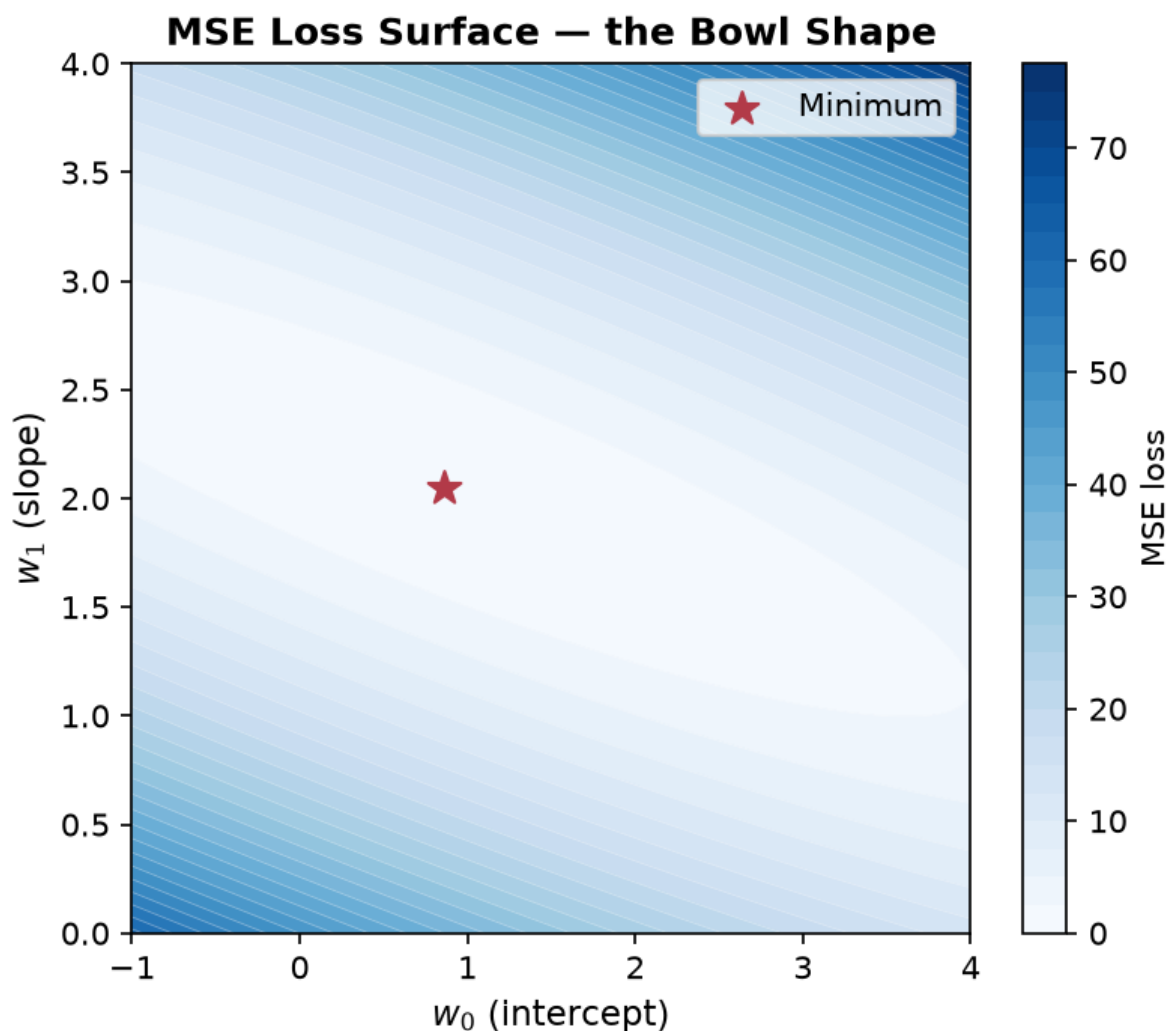


Figure 2: The MSE loss surface as a function of two weights w_0 and w_1 . The bowl shape has one minimum — the best parameters. Gradient descent finds the bottom by always stepping downhill.

⚠ Watch out

MSE is sensitive to large errors: a single badly wrong prediction can dominate the loss because of the square. For data with many outliers, consider **mean absolute error** (MAE) instead.

Worked example: Computing MSE for a simple model

Data: $(1, 2)$, $(2, 4)$, $(3, 5)$. Model: $h(x) = 0.5 + 1.5x$.

Step 1. Predictions: $h(1) = 2.0$, $h(2) = 3.5$, $h(3) = 5.0$.

Step 2. Errors: $e_1 = 2.0 - 2 = 0$, $e_2 = 3.5 - 4 = -0.5$, $e_3 = 5.0 - 5 = 0$.

Step 3. Squared errors: 0, 0.25, 0.

Step 4. $\text{MSE} = (0 + 0.25 + 0)/3 = \mathbf{0.083}$.

✓ Key takeaway

$J(\mathbf{w}) = \frac{1}{n} \sum (h(\mathbf{x}^{(i)}) - y^{(i)})^2$. We want to find the $\mathbf{w}$ that makes J as small as possible.

4 Gradient Descent

How do we find the best weights? One way is **gradient descent** (GD). The gradient of J at the current weights is a vector that points uphill. We subtract a fraction of it to step downhill. We repeat until the loss stops improving.

The intuition

Imagine you are standing on a hilly surface in fog. You want to reach the valley (minimum loss). You cannot see far, so you feel the slope under your feet and take one small step downhill. You keep stepping until the ground feels flat. That is gradient descent.

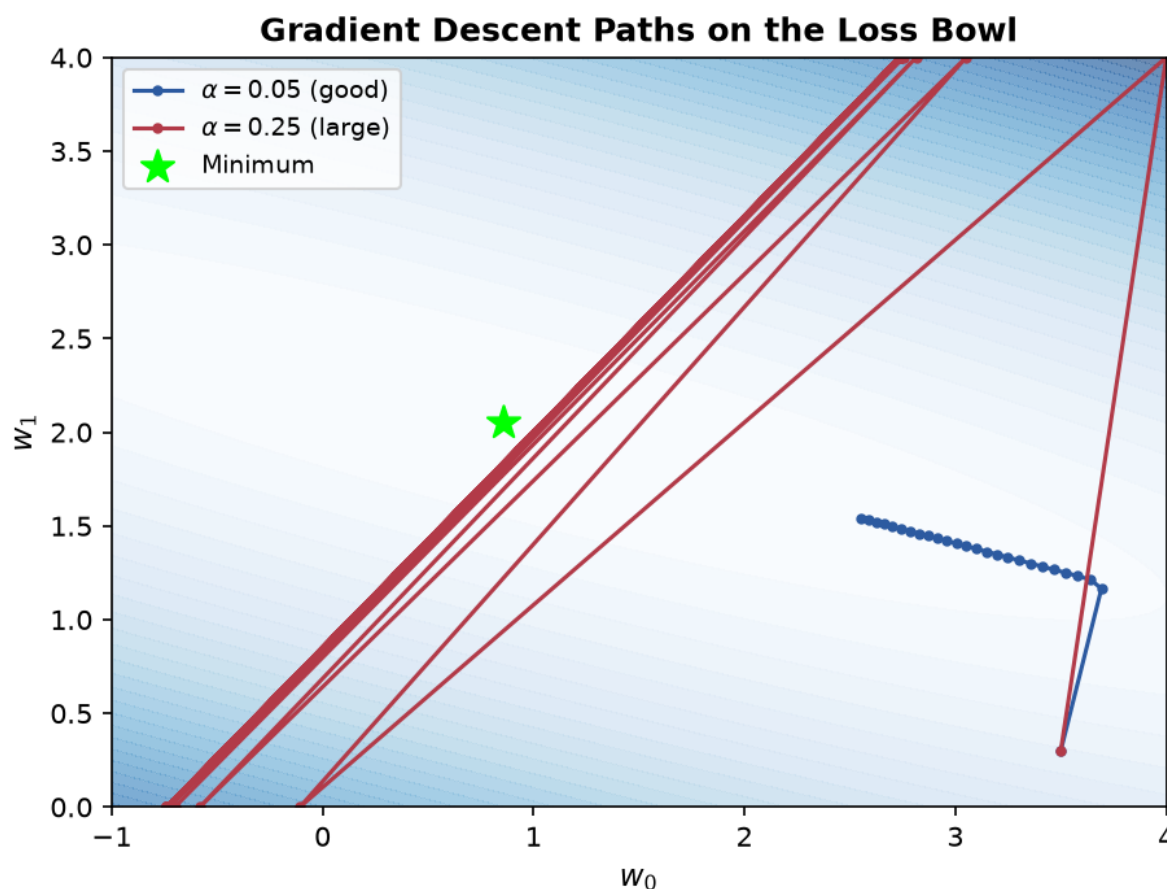


Figure 3: Gradient descent paths on the MSE loss bowl. A large learning rate (red) overshoots; a small one (blue) converges smoothly. The green star marks the minimum.

The update rule for each weight is:

$$w_j \leftarrow w_j - \frac{\alpha}{n} \sum_{i=1}^n (h(\mathbf{x}^{(i)}) - y^{(i)}) x_j^{(i)},$$

where α (alpha) is the **learning rate** — how big each step is. For the intercept w_0 we use $x_0^{(i)} = 1$ for every example.

★ Everyday picture

The learning rate is like how much you trust the slope reading under your feet. A rate of 0.01 means tiny, careful steps. A rate of 10 means giant leaps — you might jump over the valley.

Worked example: One gradient descent step for linear regression

Data: three patients from the heart disease study. Features: x_1 = diastolic pressure, x_2 = BMI. Target: y = relative risk of coronary heart disease (RR-CHD).

Patient	x_1	x_2	y
1	80	35	1.81
2	80	25	1.22
3	100	30	1.71

Initial weights: $w_0 = 5$, $w_1 = -0.03$, $w_2 = -0.03$. Learning rate $\alpha = 0.02$.

Step 1. Predict with $h(\mathbf{x}) = w_0 + w_1x_1 + w_2x_2$:

$$h_1 = 5 + (-0.03)(80) + (-0.03)(35) = 5 - 2.40 - 1.05 = 1.55$$

$$h_2 = 5 + (-0.03)(80) + (-0.03)(25) = 5 - 2.40 - 0.75 = 1.85$$

$$h_3 = 5 + (-0.03)(100) + (-0.03)(30) = 5 - 3.00 - 0.90 = 1.10$$

Step 2. Compute errors $e^{(i)} = h_i - y_i$:

$$e_1 = 1.55 - 1.81 = -0.26, \quad e_2 = 1.85 - 1.22 = +0.63, \quad e_3 = 1.10 - 1.71 = -0.61.$$

Step 3. Sum the errors and weighted errors:

$$\sum e_i = -0.26 + 0.63 - 0.61 = -0.24$$

$$\begin{aligned} \sum e_i x_{1i} &= (-0.26)(80) + (0.63)(80) + (-0.61)(100) \\ &= -20.8 + 50.4 - 61.0 = -31.4 \end{aligned}$$

$$\begin{aligned} \sum e_i x_{2i} &= (-0.26)(35) + (0.63)(25) + (-0.61)(30) \\ &= -9.10 + 15.75 - 18.30 = -11.65 \end{aligned}$$

Step 4. Update weights ($n = 3$):

$$w_0 \leftarrow 5 - \frac{0.02}{3}(-0.24) = 5 + 0.0016 = \mathbf{5.0016}$$

$$w_1 \leftarrow -0.03 - \frac{0.02}{3}(-31.4) = -0.03 + 0.2093 = \mathbf{0.1793}$$

$$w_2 \leftarrow -0.03 - \frac{0.02}{3}(-11.65) = -0.03 + 0.0777 = \mathbf{0.0477}$$

The weights moved in a direction that reduces the loss.

✗ Watch out

If α is too large, the weights oscillate or diverge. If α is too small, training takes many thousands of steps. Always plot the loss per iteration to check convergence.

✓ Key takeaway

Gradient descent: compute the gradient of the loss, subtract a fraction of it. Repeat. The learning rate α controls the step size.

5 Multiple Linear Regression

When we have $d > 1$ features, we extend the line to a **hyperplane**:

$$h(\mathbf{x}) = w_0 + w_1x_1 + w_2x_2 + \cdots + w_dx_d = \mathbf{w}^\top \tilde{\mathbf{x}},$$

where $\tilde{\mathbf{x}} = (1, x_1, \dots, x_d)$ prepends a constant 1 for the intercept. The model is still linear in the weights, even if the features are non-linear (e.g. $x_2 = (\text{age})^2$).

The intuition

SLR is a line in 2-D (one feature, one output). MLR with two features is a flat plane in 3-D. With ten features it is a hyperplane in 11-D. We cannot draw it, but the algebra is the same.

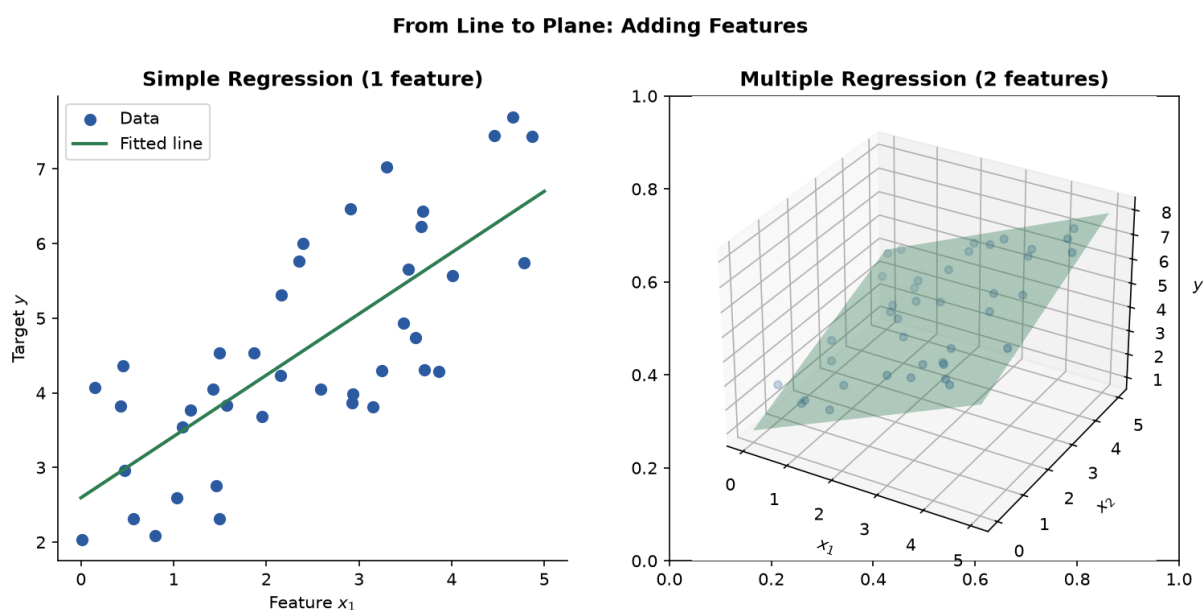


Figure 4: Left: simple regression — one feature, one line. Right: multiple regression with two features — the model is a plane. Each axis is one feature; the plane is the model's predictions.

★ Everyday picture

A doctor predicts a patient's heart-disease risk from diastolic pressure *and* BMI. Each feature pulls the prediction up or down by its weight. That is multiple linear regression.

✓ Key takeaway

MLR extends SLR to d features: $h(\mathbf{x}) = \mathbf{w}^\top \tilde{\mathbf{x}}$. The update rules for gradient descent look the same; we just have more weights to update.

6 The Normal Equation

Gradient descent is iterative. There is also a **closed-form solution** called the **normal equation**:

$$\hat{\mathbf{w}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y},$$

where $\mathbf{X}$ is the design matrix (one row per example, one column per feature plus a bias column of ones) and $\mathbf{y}$ is the target vector.

The intuition

The normal equation gives the exact answer in one shot — no iterations. It is like solving $ax = b$ by computing $x = b/a$ directly. Gradient descent is like guessing x , checking if it works, and adjusting.

⚠ Watch out

The normal equation requires inverting $\mathbf{X}^\top \mathbf{X}$, which has cost $O(d^3)$. For thousands of features it is slow. For many examples ($n \gg d$), gradient descent is preferred. If $\mathbf{X}^\top \mathbf{X}$ is singular (non-invertible), the normal equation fails — use regularisation (see later sections).

Worked example: Normal equation for two features

Use the three-patient dataset from Section 4. Design matrix (bias column first):

$$\mathbf{X} = \begin{pmatrix} 1 & 80 & 35 \\ 1 & 80 & 25 \\ 1 & 100 & 30 \end{pmatrix}.$$

Target vector:

$$\mathbf{y} = \begin{pmatrix} 1.81 \\ 1.22 \\ 1.71 \end{pmatrix}.$$

Computing $\hat{\mathbf{w}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$ gives the least-squares weights in one matrix operation. (The exact numbers require a matrix inversion; gradient descent reaches the same solution iteratively.)

✓ Key takeaway

Normal equation: $\hat{\mathbf{w}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$. One-shot, no iterations. Use it for small d ; prefer gradient descent for large d or large n .

7 Evaluating a Regression Model

After training, we need to measure how good the model is on *held-out* data (the **test set**). Three common metrics:

- **Mean absolute error** (MAE): $\text{MAE} = \frac{1}{n} \sum |h(\mathbf{x}^{(i)}) - y^{(i)}|$. Easy to interpret: “on average, we are off by this many units.”
- **Root mean squared error** (RMSE): $\text{RMSE} = \sqrt{\frac{1}{n} \sum (h(\mathbf{x}^{(i)}) - y^{(i)})^2}$. Same units as the target. Penalises large errors more than MAE.
- **R^2 score** (coefficient of determination):

$$R^2 = 1 - \frac{\sum (h(\mathbf{x}^{(i)}) - y^{(i)})^2}{\sum (\bar{y} - y^{(i)})^2},$$

where $\bar{y}$ is the mean of the targets. $R^2 = 1$ means perfect fit; $R^2 = 0$ means no better than predicting the mean every time; $R^2 < 0$ means worse than the mean.

★ Everyday picture

R^2 asks: “How much of the variability in the target did the model explain?” An $R^2 = 0.85$ means the model accounts for 85% of the spread in the data. The remaining 15% is noise the model could not capture.

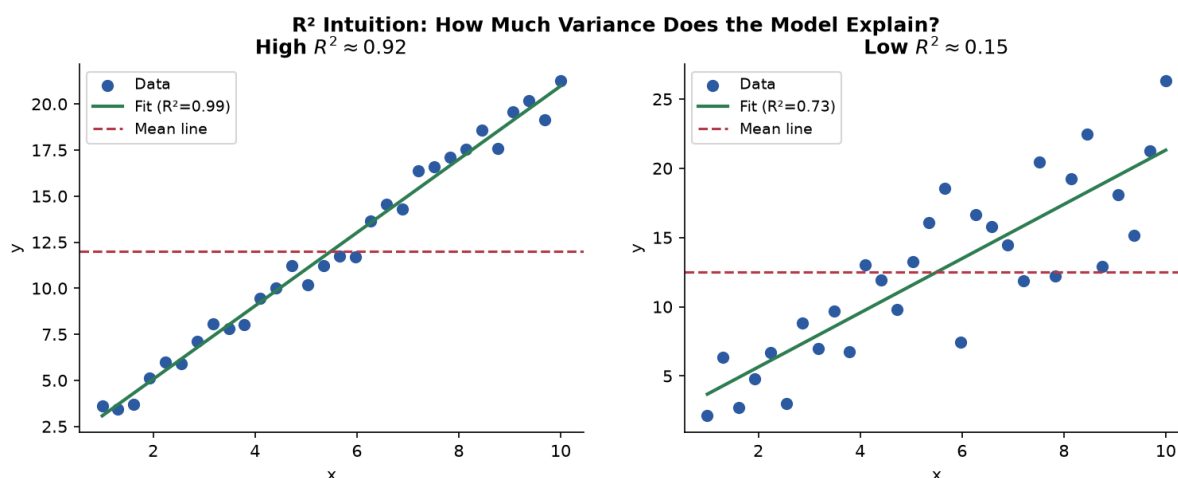


Figure 5: R^2 intuition. Left: a model with $R^2 = 0.92$ — predictions (blue) follow the true values (red) closely. Right: $R^2 = 0.15$ — the model barely beats the flat mean line.

✗ Watch out

R^2 only measures fit on the training set unless you evaluate on a test set. A model can have $R^2 = 0.99$ on training data but predict new data badly (overfitting). Always report test-set metrics.

✓ Key takeaway

Use MAE for plain error in original units, RMSE to penalise big misses, and R^2 to see how much variance the model explains. Always evaluate on a held-out test set.

8 Bias, Variance, and Overfitting

A model has two sources of error. **Bias** is the error from wrong assumptions — for example, fitting a straight line to curved data. **Variance** is the error from sensitivity to small changes in the training data.

The intuition

A model with high bias is too simple. It underfits: it misses the trend even on the training set. A model with high variance is too complex. It overfits: it memorises the training data but fails on new data. The best model balances both.

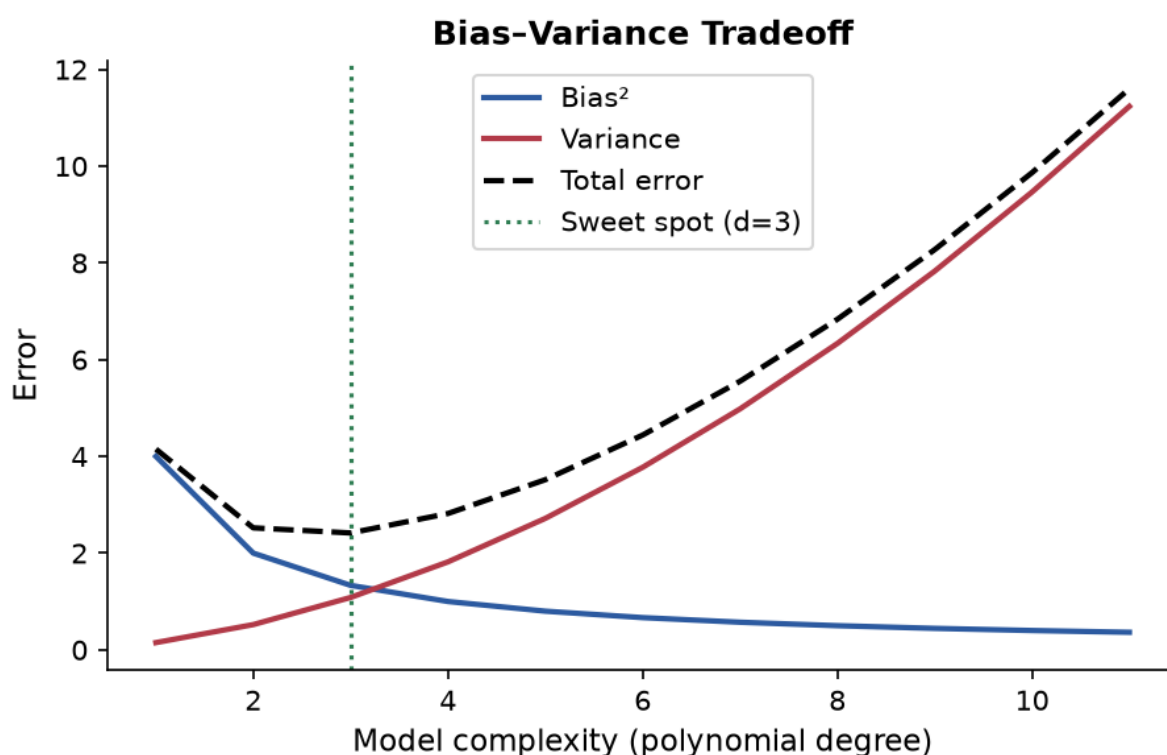


Figure 6: The bias-variance tradeoff. As model complexity grows (left to right), bias falls and variance rises. The total error (dashed) has a sweet spot — the minimum of the U-curve.

★ Everyday picture

Imagine measuring your height with a ruler. A biased ruler reads 2 cm short every time — always wrong in the same direction. A high-variance ruler gives a different answer each time you measure — noisy and inconsistent. A good ruler is accurate (low bias) and repeatable (low variance).

✗ Watch out

Adding more features always reduces training error, but it can increase test error (overfitting). Use cross-validation to find the right model complexity. Regularisation is the main tool for controlling variance.

✓ Key takeaway

Bias: the model is too simple and misses the pattern. Variance: the model is too complex and memorises the noise. Regularisation shrinks weights to lower variance at the cost of a small increase in bias.

9 Regularisation: Why We Penalise Large Weights

A model with large weights fits the training data closely. But it is fragile: a tiny change in input can cause a large change in prediction. **Regularisation** adds a penalty for large weights to the loss function. This nudges the model toward simpler, smaller-weight solutions.

The regularised loss is:

$$J_{\text{reg}}(\mathbf{w}) = \underbrace{\frac{1}{n} \sum_{i=1}^n (h(\mathbf{x}^{(i)}) - y^{(i)})^2}_{\text{data fit}} + \underbrace{\lambda \cdot \Omega(\mathbf{w})}_{\text{penalty}},$$

where λ (lambda) is the **regularisation constant** — how hard we penalise large weights — and Ω is the penalty term. We never penalise the intercept w_0 .

The intuition

The data-fit term says: “match the data”. The penalty term says: “but keep the weights small”. λ is the dial between them. A large λ makes the model prefer small weights (simpler). A small λ trusts the data more (more complex).

★ Everyday picture

Think of packing a bag for a trip. The data says: “pack everything you might need”. The weight limit (regulariser) says: “only the essentials”. λ is how strict the weight limit is.

✓ Key takeaway

Regularisation adds a penalty $\lambda \Omega(\mathbf{w})$ to the loss. It fights overfitting by keeping weights small. λ is chosen by cross-validation.

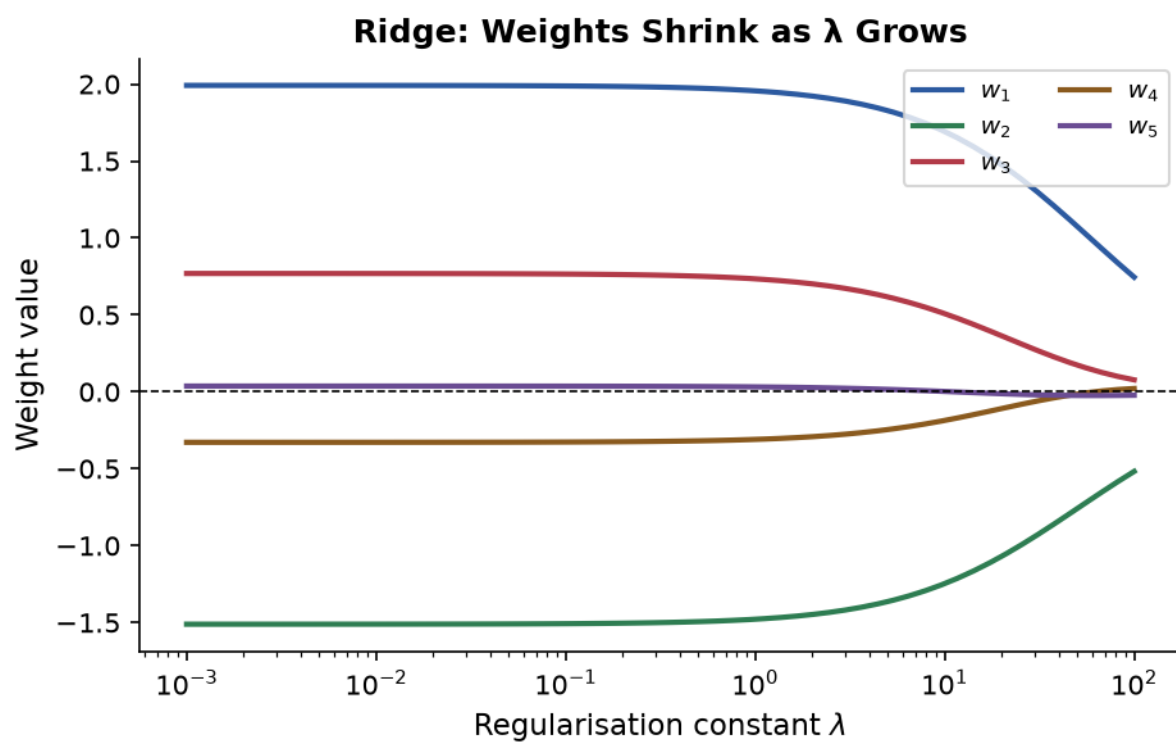


Figure 7: How regularisation shrinks weights as λ grows. At $\lambda = 0$, weights are unrestricted. As λ increases, all weights shrink toward zero.

10 Ridge Regression (L2 Regularisation)

Ridge regression uses the sum of squared weights as the penalty:

$$J_{\text{Ridge}}(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n (h(\mathbf{x}^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^d w_j^2.$$

The gradient of the penalty with respect to w_j is $2\lambda w_j$. The gradient descent update becomes:

$$w_0 \leftarrow w_0 - \frac{\alpha}{n} \sum_{i=1}^n e^{(i)},$$

$$w_j \leftarrow w_j(1 - \alpha\lambda) - \frac{\alpha}{n} \sum_{i=1}^n e^{(i)} x_j^{(i)}, \quad j \geq 1.$$

The factor $(1 - \alpha\lambda)$ multiplies w_j before the gradient step. It shrinks every weight by a fixed proportion at every step. Weights rarely reach exactly zero — they just get very small.

The intuition

Ridge pulls every weight toward zero with a spring. The stronger λ is, the stiffer the spring. All features stay in the model, but none dominates.

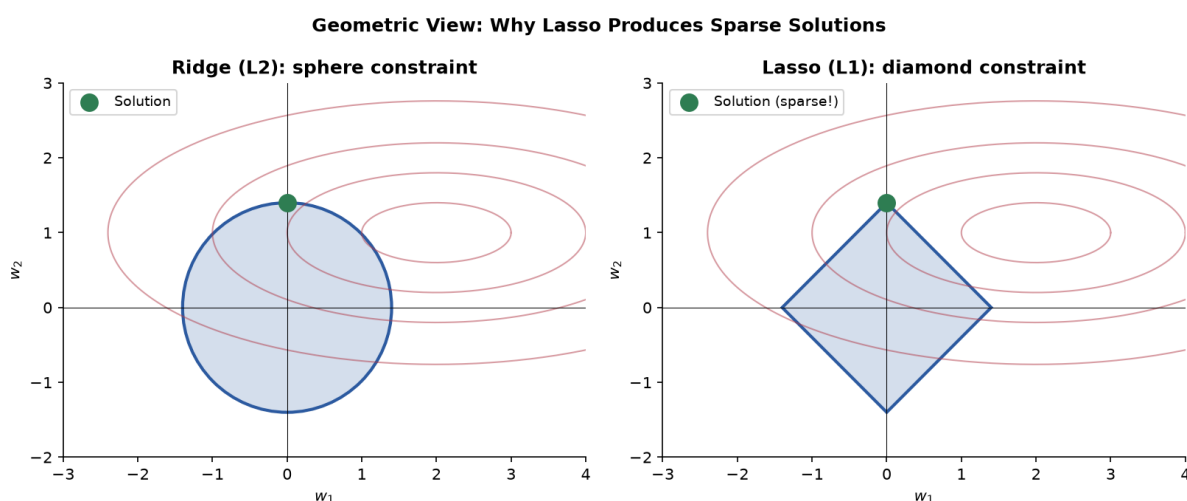


Figure 8: Geometric view of regularisation. Ridge constraint is a sphere (L2 ball); the solution lands where the loss ellipses touch the sphere. The corner of the Lasso diamond (L1 ball) often falls on an axis, driving some weights to exactly zero.

Worked example: Ridge regression: 2 iterations on the CHD dataset

Setup. Data: 3 patients, features x_1 (diastolic, mmHg) and x_2 (BMI), target y (RR-CHD). Model: $h(\mathbf{x}) = w_0 + w_1x_1 + w_2x_2$. Initial weights: $w_0 = 5$, $w_1 = w_2 = -0.03$. $\alpha = 0.02$, $\lambda = 5$. Note: $\alpha\lambda = 0.1$, so the shrinkage factor is $1 - \alpha\lambda = 0.9$.

Initial predictions and errors (same for Ridge and Lasso):

Patient	x_1	x_2	Prediction h	Target y	Error $e = h - y$
1	80	35	$5 - 0.03(80) - 0.03(35) = 1.55$	1.81	-0.26
2	80	25	$5 - 0.03(80) - 0.03(25) = 1.85$	1.22	+0.63
3	100	30	$5 - 0.03(100) - 0.03(30) = 1.10$	1.71	-0.61

Step 1. Sum errors and weighted errors:

$$\begin{aligned}\sum e_i &= -0.26 + 0.63 - 0.61 = -0.24 \\ \sum e_i x_{1i} &= (-0.26)(80) + (0.63)(80) + (-0.61)(100) = -31.4 \\ \sum e_i x_{2i} &= (-0.26)(35) + (0.63)(25) + (-0.61)(30) = -11.65\end{aligned}$$

Step 2. Ridge Iteration 1 — update weights:

$$\begin{aligned}w_0 &= 5 - \frac{0.02}{3}(-0.24) = 5 + 0.0016 = \mathbf{5.0016} \\ w_1 &= -0.03 \times 0.9 - \frac{0.02}{3}(-31.4) = -0.0270 + 0.2093 = \mathbf{0.1823} \\ w_2 &= -0.03 \times 0.9 - \frac{0.02}{3}(-11.65) = -0.0270 + 0.0777 = \mathbf{0.0507}\end{aligned}$$

Step 3. Ridge Iteration 2 — new predictions:

$$\begin{aligned}h_1 &= 5.0016 + 0.1823(80) + 0.0507(35) = 21.36 \\ h_2 &= 5.0016 + 0.1823(80) + 0.0507(25) = 20.85 \\ h_3 &= 5.0016 + 0.1823(100) + 0.0507(30) = 24.75\end{aligned}$$

Errors: $e_1 = 19.55$, $e_2 = 19.63$, $e_3 = 23.04$.

Step 4. Ridge Iteration 2 — update weights:

$$\begin{aligned}\sum e_i &= 62.22, \quad \sum e_i x_{1i} = 5438.4, \quad \sum e_i x_{2i} = 1866.2 \\ w_0 &= 5.0016 - \frac{0.02}{3}(62.22) = 5.0016 - 0.4148 = \mathbf{4.5868} \\ w_1 &= 0.1823 \times 0.9 - \frac{0.02}{3}(5438.4) = 0.1641 - 36.256 = \mathbf{-36.09} \\ w_2 &= 0.0507 \times 0.9 - \frac{0.02}{3}(1866.2) = 0.0456 - 12.441 = \mathbf{-12.40}\end{aligned}$$

Step 5. Final Ridge weights after 2 iterations: $w_0 = 4.59$, $w_1 = -36.09$, $w_2 = -12.40$.

The weights diverge because $\lambda = 5$ is very large relative to the data scale. In practice, tune λ and α using cross-validation.

⚠ Watch out

Ridge shrinks weights but never sets them to zero. All features remain in the model. If you need a sparse model (many features zeroed out), use Lasso.

✓ Key takeaway

Ridge penalty: $\lambda \sum w_j^2$. Update: multiply w_j by $(1 - \alpha\lambda)$ before the gradient step. Shrinks weights proportionally; never exactly zero.

11 Lasso Regression (L1 Regularisation)

Lasso regression (Least Absolute Shrinkage and Selection Operator) uses the sum of absolute values as the penalty:

$$J_{\text{Lasso}}(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n (h(\mathbf{x}^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^d |w_j|.$$

The gradient of $|w_j|$ is $\text{sign}(w_j)$: $+1$ if $w_j > 0$, -1 if $w_j < 0$, 0 if $w_j = 0$. The gradient descent update for $j \geq 1$ is:

$$w_j \leftarrow w_j - \frac{\alpha}{n} \sum_{i=1}^n e^{(i)} x_j^{(i)} - \alpha \lambda \cdot \text{sign}(w_j).$$

The intercept w_0 is updated as before, without a penalty term.

The intuition

Ridge pulls weights toward zero with a spring (force grows as weight grows). Lasso pushes weights toward zero with a constant nudge in the direction of the axis. A constant push can push small weights all the way to zero, removing features entirely. This is **automatic feature selection**.

Worked example: Lasso regression: 2 iterations on the CHD dataset

Same setup as the Ridge example above. Initial predictions and errors are identical.

Step 1. Lasso Iteration 1. Since $w_1 = w_2 = -0.03 < 0$: $\text{sign}(w_1) = \text{sign}(w_2) = -1$.
Note: $\alpha\lambda = 0.1$.

$$\begin{aligned} w_0 &= 5 - \frac{0.02}{3}(-0.24) = \mathbf{5.0016} \\ w_1 &= -0.03 - \frac{0.02}{3}(-31.4) - 0.1(-1) = -0.03 + 0.2093 + 0.1 = \mathbf{0.2793} \\ w_2 &= -0.03 - \frac{0.02}{3}(-11.65) - 0.1(-1) = -0.03 + 0.0777 + 0.1 = \mathbf{0.1477} \end{aligned}$$

Step 2. Lasso Iteration 2 — new predictions:

$$\begin{aligned} h_1 &= 5.0016 + 0.2793(80) + 0.1477(35) = 32.52 \\ h_2 &= 5.0016 + 0.2793(80) + 0.1477(25) = 31.04 \\ h_3 &= 5.0016 + 0.2793(100) + 0.1477(30) = 37.36 \end{aligned}$$

Errors: $e_1 = 30.71$, $e_2 = 29.82$, $e_3 = 35.65$.

Step 3. Lasso Iteration 2 — update weights. Now $w_1 = 0.2793 > 0$ and $w_2 = 0.1477 > 0$: $\text{sign}(w_1) = \text{sign}(w_2) = +1$.

$$\sum e_i = 96.18, \quad \sum e_i x_{1i} = 8407.4, \quad \sum e_i x_{2i} = 2889.85$$

$$\begin{aligned} w_0 &= 5.0016 - \frac{0.02}{3}(96.18) = 5.0016 - 0.6412 = \mathbf{4.3604} \\ w_1 &= 0.2793 - \frac{0.02}{3}(8407.4) - 0.1(+1) = 0.2793 - 56.049 - 0.1 = \mathbf{-55.87} \\ w_2 &= 0.1477 - \frac{0.02}{3}(2889.85) - 0.1(+1) = 0.1477 - 19.266 - 0.1 = \mathbf{-19.22} \end{aligned}$$

Step 4. Final Lasso weights after 2 iterations: $w_0 = 4.36$, $w_1 = -55.87$, $w_2 = -19.22$.

Both methods diverge here because $\lambda = 5$ is very large relative to the feature scale. In practice, scale the features to $[0, 1]$ or z -score them, and tune λ by cross-validation.

✗ Watch out

Lasso can set weights to exactly zero, which Ridge cannot. But Lasso is not differentiable at $w_j = 0$, so standard gradient descent uses the sub-gradient (sign) or coordinate descent instead. With many correlated features, Lasso picks one arbitrarily and zeroes the rest.

✓ Key takeaway

Lasso penalty: $\lambda \sum |w_j|$. Update: subtract $\alpha \lambda \text{sign}(w_j)$ — a constant nudge. Can drive weights exactly to zero; removes features from the model.

12 Ridge vs. Lasso: Side-by-Side Comparison

Property	Ridge (L2)	Lasso (L1)
Penalty term	$\lambda \sum w_j^2$	$\lambda \sum w_j $
Gradient of penalty	$2\lambda w_j$ (smooth)	$\lambda \text{sign}(w_j)$ (non-smooth)
GD update	multiply by $(1 - \alpha\lambda)$	subtract $\alpha\lambda \text{sign}(w_j)$
Shrinkage type	proportional (large weights shrink more)	fixed step (constant push)
Can set weights to zero?	Rarely (only at $w_j = 0$ exactly)	Yes
Feature selection?	No	Yes
Geometry	Ellipse touches sphere	Ellipse touches diamond
Best when	All features relevant	Many features irrelevant

★ Everyday picture

Ridge is like tightening a budget evenly: everyone's spending is cut by the same percentage. Lasso is like telling some people their budget is exactly zero: those items are cut completely.

✓ Key takeaway

Use Ridge when you believe all features matter and want smooth shrinkage. Use Lasso when you want the model to pick a small subset of features. Elastic Net combines both penalties and is useful when features are correlated.

13 Choosing the Regularisation Constant λ

λ is a **hyperparameter** — we do not learn it from the data; we choose it before training. The right way to choose λ is **k -fold cross-validation**:

1. Pick a list of candidate values, e.g. $\lambda \in \{0.001, 0.01, 0.1, 1, 5, 10\}$.
2. For each λ , split the training data into k folds.
3. Train on $k - 1$ folds, validate on the remaining fold.
4. Average the validation error over all k splits.
5. Pick the λ with the lowest average validation error.

⚠ Watch out

Never pick λ by looking at the test set. That leaks test information into the model and gives an overly optimistic estimate of future performance.

✓ Key takeaway

Tune λ with cross-validation. Scale your features before regularising, so λ treats all features equally. Never use the test set to choose λ .